Assignment 3 Solution

Muhammad Saad Khan

253309919

CS 3340

Question 1: Suffix Array for "boboobytippohytippih"

List all suffixes indexed from position 0 to 19, then sort them lexicographically. The suffix array is the sequence of starting indices in sorted order.

All Suffixes

Index	Suffix
0	boboobytippohytippih
1	oboobytippohytippih
2	boobytippohytippih
3	oobytippohytippih
4	obytippohytippih
5	bytippohytippih
6	ytippohytippih
7	tippohytippih
8	ippohytippih
9	ppohytippih
10	pohytippih
11	ohytippih
12	hytippih
13	ytippih
14	tippih
15	ippih
16	ppih
17	pih
18	ih
19	h

Sorted Suffixes (Suffix Array)

Rank	Index	Suffix

0	5	bytippohytippih
1	0	boboobytippohytippih
2	2	boobytippohytippih
3	19	h
4	12	hytippih
5	18	ih
6	8	ippohytippih
7	15	ippih
8	11	ohytippih
9	4	obytippohytippih
10	1	oboobytippohytippih
11	3	oobytippohytippih
12	10	pohytippih
13	17	pih
14	9	ppohytippih
15	16	ppih
16	7	tippohytippih
17	14	tippih
18	6	ytippohytippih
19	13	ytippih

Suffix Array: `[5, 0, 2, 19, 12, 18, 8, 15, 11, 4, 1, 3, 10, 17, 9, 16, 7, 14, 6, 13]`

Question 2: Unbounded Knapsack Problem

1. Algorithm Description

Use dynamic programming. Define dp[w] as the maximum value achievable with capacity w. Initialize dp[0] = 0 and dp[w] = 0 for all w from 1 to W. Then for each capacity w from 1 to W, iterate over all n items. For each item i, if w_i <= w, update:

```
dp[w] = max(dp[w], dp[w - w_i] + v_i)
```

Unlike the 0/1 knapsack, capacity is iterated in increasing order (not decreasing), which allows the same item to be reused, because dp[w - w_i] may already include item i.

2. Correctness

The recurrence dp[w] = max over all i where w_i <= w of (dp[w - w_i] + v_i) is correct because at any capacity w, the optimal solution either uses none of the items or uses some item i last, leaving capacity w - w_i filled optimally. Since items can be reused, dp[w - w_i] is allowed to include item i again. By induction: dp[0] = 0 is the correct base case, and if dp[0..w-1] are all correct, then dp[w] is correctly computed as the best over all possible last items chosen.

3. Complexity

The outer loop runs W times, and the inner loop runs n times, giving **O(nW)** time and **O(W)** space.

Question 3: Maximum Spanning Tree

1. Algorithm Description

Take Prim's MST algorithm and reverse the selection criterion. Instead of always selecting the minimum weight edge crossing the cut, select the maximum weight edge. Maintain the same heap structure but initialize all keys to -infinity instead of +infinity and replace the decrease_key condition with increase_key updating a vertex's key when a heavier edge to it is found. The rest of the algorithm is identical to Prim's.

2. Correctness

The correctness follows directly from the Cut Property for maximum spanning trees: for any cut of the graph, the maximum weight edge crossing the cut belongs to some maximum spanning tree. Prim's algorithm always selects the extremal (min or max) edge crossing the current cut between visited and unvisited vertices. By choosing the maximum instead of the minimum at every step, the same greedy argument holds no other spanning tree can have greater total weight, because at each step we make the locally optimal choice that is also globally optimal by the cut property.

3. Complexity

Identical to Prim's MST: **O ((|V| + |E|) log |V|)** using a binary heap.

Question 4: Counterexample for Dijkstra with Negative Edges

Consider a graph with three vertices: s, u, v and the following edges:

Edge	Weight
`s -> u`	`2`
`s -> v`	`4`
`u -> v`	`-3`

Dijkstra's Execution

Step	Action	dist[s]	dist[u]	dist[v]
1	`Initialize at s`	`0`	`inf`	`inf`
2	`Relax s's edges`	`0`	`2`	`4`
3	`Extract u (dist=2), relax u->v`	`0`	`2`	`4 (not updated)`
4	`Extract v (dist=4), finalized`	`0`	`2`	`4 (WRONG)`

Correct shortest path: `s -> u -> v = 2 + (-3) = -1`

Dijkstra's answer: `dist[v] = 4` **(incorrect)**

Dijkstra greedily finalizes dist[v] = 4 via the direct edge before discovering the cheaper path through the negative edge u -> v. Once v is extracted from the priority queue it is considered finalized and will not be updated, producing an incorrect result.

Question 5: Minimum Weight Cycle using Floyd-Warshall

1. Algorithm Description

Use the Floyd-Warshall all-pairs shortest path algorithm. Define dist[i][j] as the shortest path distance from vertex i to vertex j. Initialize as follows:

```
dist[i][j] = weight(i, j)   if edge (i, j) exists
dist[i][i] = 0              for all i
dist[i][j] = infinity       otherwise
```

Run the standard Floyd-Warshall relaxation for all intermediate vertices k from 1 to |V|:

```
dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

After the algorithm completes, the minimum weight cycle is found by taking the minimum value of dist[i][i] over all vertices i. The diagonal entries, after relaxation, represent the shortest path from i back to itself i.e. a cycle. The minimum over all dist[i][i] gives the weight of the minimum weight cycle in G.

2. Correctness

After Floyd-Warshall completes, dist[i][j] holds the weight of the shortest path from i to j using any intermediate vertices. In particular, dist[i][i] after relaxation represents the shortest path that starts at i, passes through some other vertices, and returns to i which is exactly a cycle through i. Since we compute this for all vertices i and take the minimum, we find the minimum weight cycle in the graph. The graph has no negative cycles by assumption, so Floyd-Warshall is valid and all distances are finite or infinity.

3. Complexity

Floyd-Warshall runs three nested loops each of size |V|, giving **O(|V|^3)** time and **O(|V|^2)** space for the distance matrix exactly meeting the required bound.